

챕터 4·5 환경설정 개념 가이드

1. 전체 구조 — 지금 내 PC는 어떤 상태인가?
2. 설치 과정에서 한 일의 의미
3. WSL이란? — Windows 안의 Linux
 - WSL이 필요한 이유
 - WSL 1과 WSL 2의 차이
 - Ubuntu란?
 - WSL Integration 체크가 하는 일
4. Docker란? — 환경을 통째로 패키징하는 도구
 - Docker가 필요한 이유
 - 핵심 개념 정리
 - 설정 파일의 역할
 - 볼륨(Volume) — 파일이 사라지지 않는 이유
5. NVIDIA CUDA — GPU를 쓸 수 있는 이유
 - GPU가 왜 필요한가?
 - CUDA란?
 - 드라이버와 CUDA는 어디에 설치되어 있나?
 - 정상 상태 확인 방법
6. JupyterLab이란? — 브라우저 기반 코드 편집기
 - JupyterLab의 특징
 - 접속 정보
7. 실습 환경 안에는 무엇이 들어있나?
8. 전체 흐름 요약
9. 용어 사전

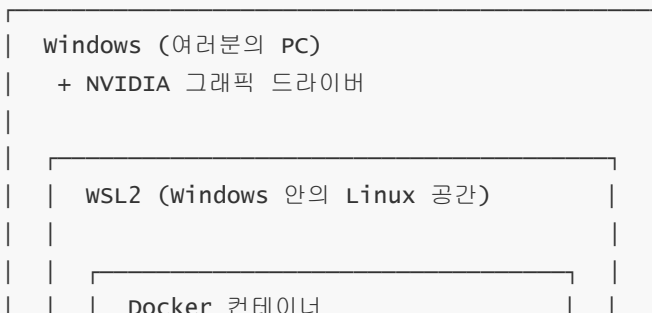
챕터 4·5 환경설정 개념 가이드

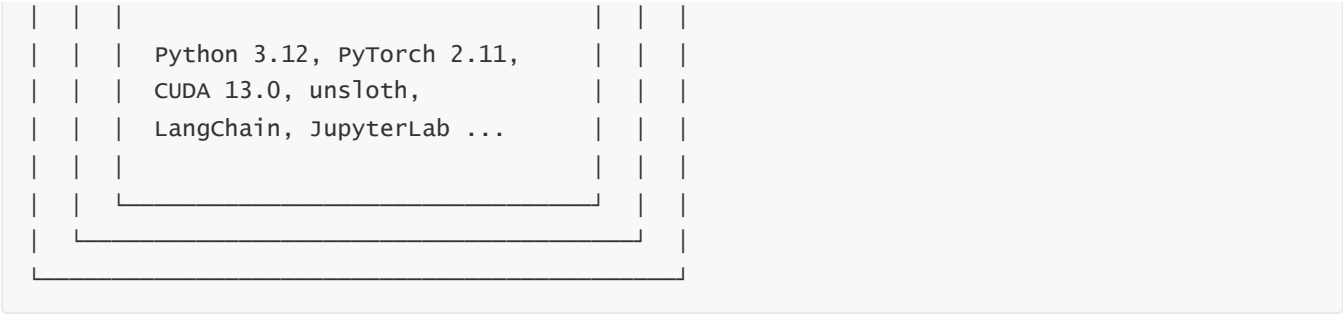
이 문서는 뭔가요?

환경설정 가이드를 따라 NVIDIA 드라이버, Docker Desktop, WSL을 설치하셨을 겁니다. 이 문서는 "그래서 지금 내 PC에 뭘 설치한 건지", "왜 이런 과정이 필요한 건지"를 정리한 **참고 문서**입니다. 설치 절차가 아니라 **개념과 이유**에 초점을 맞추고 있습니다.

1. 전체 구조 — 지금 내 PC는 어떤 상태인가?

환경설정을 모두 마친 여러분의 PC는 아래와 같은 구조로 되어 있습니다.





정리하면 이런 관계입니다.

- **NVIDIA 드라이버** — Windows에 설치. GPU를 WSL2와 컨테이너에 빌려주는 역할
- **WSL2** — Windows 안에 만들어진 Linux 공간
- **Docker** — 그 Linux 공간 위에서 실습 환경을 통째로 패키징한 것
- **JupyterLab** — Docker 컨테이너 안에서 실행되는 코드 편집기

중요한 하나: Python, PyTorch, CUDA는 **여러분의 Windows가 아니라 컨테이너 안에** 설치되어 있습니다. Windows에는 아무것도 깔지 않았습니

2. 설치 과정에서 한 일의 의미

순서	한 일	의미
1	NVIDIA 그래픽 드라이버 설치	Windows가 GPU를 인식하고, 그 GPU를 WSL2·컨테이너에 빌려 줄 수 있게 만든 것
2	Docker Desktop 설치	컨테이너를 만들고 관리하는 도구를 설치한 것
3	WSL 설치 (<code>wsl --install</code>)	Windows 안에 Linux(Ubuntu) 공간을 만든 것. 재시작이 필요한 이유는 Windows의 가상화 기능을 켜야 하기 때문 (반드시 Powershell 을 관리자 권한으로 실행해야 합니다.)
4	Docker Desktop => Resources => WSL integration 체크	Docker와 WSL을 연결한 것. 이걸 켜야 WSL 안에서 Docker가 동작함
5	<code>docker compose up --build</code>	설계도(Dockerfile)를 읽어 실습 환경 이미지를 실제로 만든 것. 처음 한 번만 오래 걸림
6	<code>verify_env.sh</code> 실행	만들어진 환경이 정상인지 자동 점검한 것

💡 왜 재시작을 했나요?

WSL2는 Windows의 가상화 기능(Virtual Machine Platform) 위에서 동작합니다.
이 기능은 Windows 부팅 시점에 켜지기 때문에, 활성화 후 반드시 재시작이 필요합니다.

3. WSL이란? — Windows 안의 Linux

WSL이 필요한 이유

AI/ML 생태계의 핵심 라이브러리(PyTorch, HuggingFace, LangChain 등)는 **Linux 환경을 기준으로 개발**되어 있습니다. Windows에서 직접 설치하면 경로 문제, 인코딩 문제, 빌드 실패 등 예상치 못한 오류가 자주 발생합니다.

WSL(Windows Subsystem for Linux)은 Windows 안에 Linux를 설치할 수 있게 해주는 기능입니다. 별도의 컴퓨터 없이도 Linux 환경을 사용할 수 있습니다.

WSL 1과 WSL 2의 차이

구분	WSL 1	WSL 2
동작 방식	Linux 명령어를 Windows가 번역	실제 Linux 커널을 실행
성능	상대적으로 느림	빠름 (네이티브에 가까움)
Docker 호환	제한적	완벽 지원
GPU 공유	불가	가능

챕터 4, 5에서 GPU를 활용한 AI 모델 학습을 진행하기 때문에 **WSL 2**를 사용합니다. `wsl --install` 은 기본으로 WSL 2를 설치합니다.

Ubuntu란?

Ubuntu는 Linux 배포판 중 하나입니다. Linux에는 다양한 "맛(배포판)"이 있는데, Ubuntu는 그중 커뮤니티가 가장 크고 학습 자료가 가장 풍부합니다. 실무에서도 서버 환경으로 가장 많이 사용됩니다.

`wsl --install` 을 실행하면 Ubuntu가 **자동으로 함께 설치**됩니다. 따로 고를 필요가 없습니다.

WSL Integration 체크가 하는 일

4단계에서 켜 **WSL integration** 옵션이 하는 일은 이렇습니다.

[체크 안 함]	Ubuntu 안에서 docker 명령 => "command not found"
[체크 함]	Ubuntu 안에서 docker 명령 => Docker Desktop이 응답

Docker Desktop은 Windows 앱이지만, 실제 컨테이너는 WSL2 안에서 돌립니다. 이 옵션은 그 둘을 이어주는 다리입니다. **이 체크를 빠뜨리면 5단계에서 반드시 실패**하므로, 설치가 안 될 때 가장 먼저 확인해야 할 항목입니다.

4. Docker란? — 환경을 통째로 패키징하는 도구

Docker가 필요한 이유

AI 실습 환경을 구성하려면 Python, PyTorch, CUDA, LangChain 등 수백 개의 라이브러리를 **정확한 버전 조합**으로 설치해야 합니다. 그런데 교육생이 각자 다른 PC에서 작업하면:

- Python 버전이 다르거나
- CUDA 버전이 맞지 않거나
- 라이브러리 간 버전이 충돌하거나

하는 문제가 빈번하게 생깁니다.

Docker는 **운영체제, 라이브러리, 설정 파일을 하나의 패키지(이미지)로 묶어서** 누구의 PC에서 열어도 동일한 환경을 보장하는 도구입니다. 실무에서도 개발 환경 통일, 서비스 배포, CI/CD 파이프라인 등에 폭넓게 사용됩니다.

핵심 개념 정리

용어	설명	비유
이미지 (Image)	실습 환경의 설계도이자 완성품. 어떤 OS, 어떤 라이브러리를 쓸지 정의되어 있음	밀키트
컨테이너 (Container)	이미지를 기반으로 실제로 실행된 환경. 우리가 작업하는 공간	밀키트로 차린 실제 밥상
Docker Desktop	이미지와 컨테이너를 관리하는 GUI 앱	주방 관리 대시보드

하나의 이미지로 여러 개의 컨테이너를 만들 수 있고, 컨테이너를 삭제해도 이미지는 남아있으므로 언제든 다시 만들 수 있습니다.

설정 파일의 역할

실습 폴더에 있는 설정 파일들이 각각 어떤 역할을 하는지 정리합니다.

파일	역할
Dockerfile	이미지 빌드 규칙. 베이스 이미지, Python 버전, 패키지 설치 순서가 정의되어 있음
docker-compose.yaml	컨테이너 실행 방식. 포트 번호, 폴더 연결(볼륨), GPU 할당, 환경변수를 설정
requirements.lock.txt	실제 설치되는 Python 패키지 목록. 250개 넘는 패키지의 버전이 전부 고정되어 있음
requirements.linux.txt	강사가 패키지를 추가·변경할 때 사용하는 원본 목록 (빌드에는 쓰이지 않음)
scripts/verify_env.sh	환경이 정상 구성되었는지 자동 점검하는 스크립트

💡 왜 requirements.txt 가 아니라 requirements.lock.txt 인가요?

일반 requirements.txt 는 "이 정도 버전이면 된다"는 느슨한 요구사항입니다.
그래서 설치하는 날짜에 따라 버전이 달라질 수 있습니다.

.lock 파일은 "정확히 이 버전들"이라고 전부 못 박은 목록입니다.

덕분에 오늘 설치하든 다음 달에 설치하든 모두가 동일한 환경을 얻습니다.

실무에서도 package-lock.json, poetry.lock, uv.lock 등 같은 개념을 사용합니다.

볼륨(Volume) — 파일이 사라지지 않는 이유

Docker 컨테이너는 기본적으로 임시 공간입니다. 컨테이너를 삭제하면 내부 데이터도 함께 사라집니다. 하지만 docker-compose.yaml 에서 볼륨 마운트를 설정해두었기 때문에, JupyterLab에서 작업한 파일은 Windows의 실습 폴더와 실시간으로 동기화됩니다.

즉, 컨테이너를 Stop하거나 Down해도 작업 파일은 Windows 폴더에 그대로 남아있습니다.

반대로 말하면, 컨테이너 안에서 pip install 로 설치한 패키지는 볼륨 밖에 있으므로 사라집니다. 이 구분이 실습 중 가장 자주 헷갈리는 지점입니다.

5. NVIDIA CUDA — GPU를 쓸 수 있는 이유

GPU가 왜 필요한가?

AI 모델의 학습과 추론에는 대량의 행렬 연산이 필요합니다. CPU는 범용 연산에 강하지만, 이런 병렬 연산에는 GPU가 압도적으로 빠릅니다.

비교 항목	CPU	GPU
코어 수	수 개 ~ 수십 개	수천 개
특기	복잡한 순차 연산	단순한 병렬 연산 (행렬 곱 등)
AI 학습 속도	느림	수십~수백 배 빠름

CUDA란?

CUDA(Compute Unified Device Architecture)는 NVIDIA가 만든 GPU 프로그래밍 플랫폼입니다. PyTorch 같은 AI 프레임워크가 GPU를 사용하려면 CUDA가 필요합니다.

드라이버와 CUDA는 어디에 설치되어 있나?

물론 우리는 글로벌에서 CUDA를 미리 설치했습니다.

구성 요소	설치 위치	이유
NVIDIA 드라이버	Windows	GPU 하드웨어를 직접 제어. 반드시 호스트에 있어야 함
CUDA 런타임 라이브러리	Docker 컨테이너 안	이미지에 이미 포함되어 있음. 여러분이 설치한 게 아님

즉 GPU를 쓰기 위해 Windows에 반드시 필요한 것은 드라이버 하나입니다. 컨테이너가 CUDA 13.0을 자체적으로 들고 있기 때문에, 버전 충돌 걱정 없이 어떤 PC에서도 동일하게 동작합니다.

정상 상태 확인 방법

Windows PowerShell에서 `nvidia-smi`를 실행했을 때 아래 정보가 표로 출력되면 드라이버는 정상입니다.

- GPU 이름 (예: NVIDIA GeForce RTX 5060 Ti)
- Driver Version
- CUDA Version — 드라이버가 지원하는 최대 CUDA 버전을 의미합니다. 이 값이 13.0 이상이어야 실습 환경이 동작합니다.

컨테이너 쪽 확인은 `verify_env.sh`가 자동으로 해줍니다. 직접 확인하고 싶다면 JupyterLab에서:

```
import torch

print('cuda available:', torch.cuda.is_available()) # True가 나와야 정상
print('gpu:', torch.cuda.get_device_name(0))
```

6. JupyterLab이란? — 브라우저 기반 코드 편집기

JupyterLab의 특징

JupyterLab은 브라우저에서 동작하는 대화형 개발 환경입니다. 본 과정에서 기본 IDE로 사용합니다.

특징	설명
셀 단위 실행	코드를 작은 단위로 나누어 한 셀씩 실행하고 결과를 바로 확인
시각화 지원	그래프, 표, 이미지 등을 코드 바로 아래에 출력
마크다운 지원	코드와 설명을 하나의 노트북 파일(.ipynb)에 함께 작성 가능
별도 설치 불필요	Docker 컨테이너 안에 이미 설치되어 있어 브라우저만 열면 사용 가능

접속 정보

항목	값
URL	http://127.0.0.1:8888/lab
Token	lab

http://127.0.0.1 는 "내 컴퓨터"를 의미하고, 8888 은 JupyterLab이 사용하는 포트 번호입니다.
Docker 컨테이너 내부의 8888번 포트가 여러분 PC의 8888번 포트와 연결되어 있어서,
브라우저에서 이 주소로 접속하면 컨테이너 안의 JupyterLab에 도달하게 됩니다.

7. 실습 환경 안에는 무엇이 들어있나?

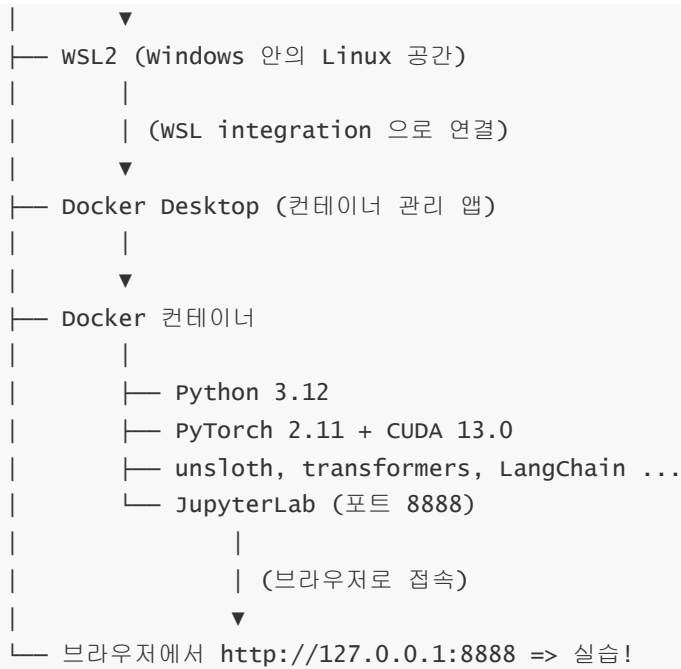
verify_env.sh 가 점검하는 항목이 곧 이 환경의 핵심 구성입니다.

계층	주요 패키지	용도
기반	Python 3.12, PyTorch 2.11, CUDA 13.0	연산 엔진
파인튜닝	unsloth, peft, trl, bitsandbytes	LLM 학습·양자화 (챕터 5)
모델/데이터	transformers, datasets, accelerate	모델 로딩과 데이터 처리
RAG/에이전트	langchain, langgraph, chromadb, openai	챕터 4
분석/시각화	pandas, numpy, matplotlib, scikit-learn	데이터 처리와 결과 확인
작업 환경	JupyterLab	코드 편집기

💡 bitsandbytes 와 4bit 양자화
8GB급 GPU에서도 큰 모델을 다룰 수 있는 이유입니다.
모델의 가중치를 16비트 대신 4비트로 압축해 메모리 사용량을 크게 줄입니다.
자세한 내용은 챕터 5에서 다룹니다.

8. 전체 흐름 요약





9. 용어 사전

용어	설명
WSL	Windows Subsystem for Linux. Windows 안에서 Linux를 실행할 수 있게 해주는 기능
WSL integration	Docker Desktop과 WSL을 연결하는 설정. 커야 Ubuntu 안에서 docker 명령을 쓸 수 있음
Ubuntu	Linux 배포판 중 하나. 가장 널리 사용되며 커뮤니티 자료가 풍부함
Docker	애플리케이션과 그 실행 환경을 하나로 패키징하는 컨테이너 플랫폼
이미지	Docker 컨테이너의 설계도이자 완성품. 실행에 필요한 모든 것이 정의되어 있음
컨테이너	이미지를 기반으로 실제 실행된 인스턴스. 우리가 작업하는 공간
빌드 (build)	Dockerfile을 읽어 이미지를 만드는 과정. 처음 한 번만 오래 걸림
볼륨	컨테이너와 Windows 폴더를 연결하는 기능. 파일 동기화에 사용
lock 파일	패키지 버전을 전부 고정한 목록. 재현 가능한 환경을 만들기 위한 장치
CUDA	NVIDIA GPU를 프로그래밍에 활용할 수 있게 해주는 플랫폼
nvidia-smi	GPU 상태를 확인하는 명령어 (System Management Interface)
JupyterLab	브라우저 기반 대화형 개발 환경. 셀 단위 코드 실행이 가능
PowerShell	Windows의 명령줄 도구. Docker 명령어를 여기서 실행

용어	설명
포트 (Port)	네트워크 통신의 출입구 번호. JupyterLab은 8888번 포트를 사용
Token	JupyterLab 접속 시 사용하는 인증 키. 본 과정에서는 <code>1ab</code> 으로 설정됨
양자화 (Quantization)	모델 가중치의 정밀도를 낮춰 메모리 사용량을 줄이는 기법 (예: 4bit)
PEP8	Python 코드 작성 스타일 가이드. 가독성 높은 코드 작성을 위한 규칙

💡 이 문서는 필요할 때 돌아와서 참고하는 용도입니다.

모든 내용을 한 번에 외울 필요는 없으며, 실습을 진행하면서 자연스럽게 익숙해집니다.